Existing compilers miss useful optimizations

Existing optimization passes are incomplete.
Useful transformations can be absent or fail to trigger.

Incomplete rewrite coverage

Pattern-based optimizers cannot cover every useful instruction sequence.

Conservative decisions

Legality or cost analysis may not establish the conditions needed to apply a rewrite.

Pipeline interactions

Pass ordering and lowering can expose a pattern after the relevant optimization has run.

A missed optimization has a legal, beneficial alternative that the selected compiler pipeline does not produce for the case and target.

Motivation: LPO introduction. Mature optimizers continue to gain new and generalized rewrite patterns.

[REWORK] How optimization opportunities reach engineers today

Current mechanism	Trigger and engineering work
Nightly performance regression testing	Detect deterioration against a baseline before upstream integration. Investigate or revert the offending change.
Performance issue tickets	Investigate slow cases discovered in already merged code.
Kernel-specific tasks	Inspect generated code and profiling data to make a specified kernel faster.
Optimization epics	Proactively explore an optimization class, usually in response to an observed performance gap.

The gap: routine discovery of optimization opportunities before a performance problem is noticed. (To my knowledge)

Routine discovery needs a repeatable investigation process



Representative repeated investigation activities

Work	Material an engineer needs
Understand the region	Definitions, uses, effects, aliasing, shape/layout and numeric constraints.
Test an alternative	Compiler entry point, enclosing module, target settings, harness and inputs.
Assess the result	Comparable output artifacts, selected metric, conditions and reproduction steps.

Build a framework for reproducible optimization discovery

Develop a framework that searches for beneficial MLIR rewrites and returns a reproducible original / candidate comparison for AscendNPU-IR.



Offload variant construction, repeated checks and reproduction packaging; measure whether the resulting findings justify preparation and review effort.

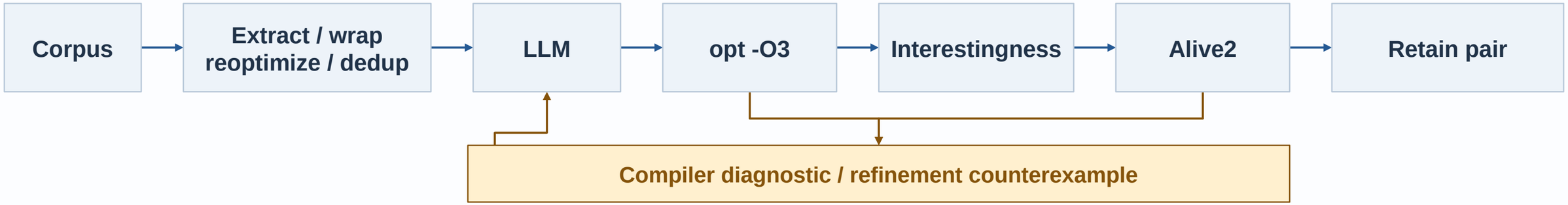
Research directions and concrete starting points

Study	Concrete starting points	What to compare	Evidence to retain
Improvement metrics	MLIR op statistics. LLVM TTI / llvm-mca. Ascend msProf.	Cheap indicators vs downstream code or measured execution.	Coverage, later-pass survival and collection cost.
Compilation / checks	MLIR verifier / Alive2. MLIR-TV / xdsl-smt.	Checkpoint compatibility and semantic coverage.	Route matrix and reproducible probes.
Agent context	Codex / Claude Code / OpenCode. SQL agents / PaperQA.	Direct dialect context vs retrieval or added analyses.	Useful findings, invalid edits and context cost.
Agent tools	File edits and compiler diagnostics. Operation-aware editor.	Unrestricted edits vs scoped IR operations and check tools.	Valid proposals, reachable rewrites and total cost.
Region discovery	Use-def / effect scans. Available cost or profile estimates.	Supplied range vs prompted, static or cost-guided selection.	Region coverage, useful yield and selection cost.
Feedback / policy	LPO errors / counterexamples. PRAGMA profiles and history.	Feedback detail, continuation and budget allocation.	Attempt traces, useful yield and expensive checks.

Starting points to investigate. Availability and useful coverage for AscendNPU-IR remain to be demonstrated.

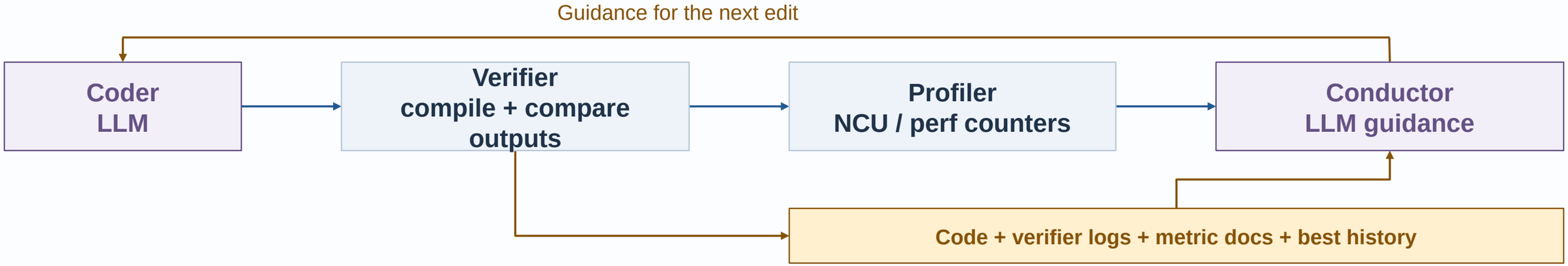
Two paper workflows inform the experiment

LPO: Discovering Missed Peephole Optimizations with Large Language Models



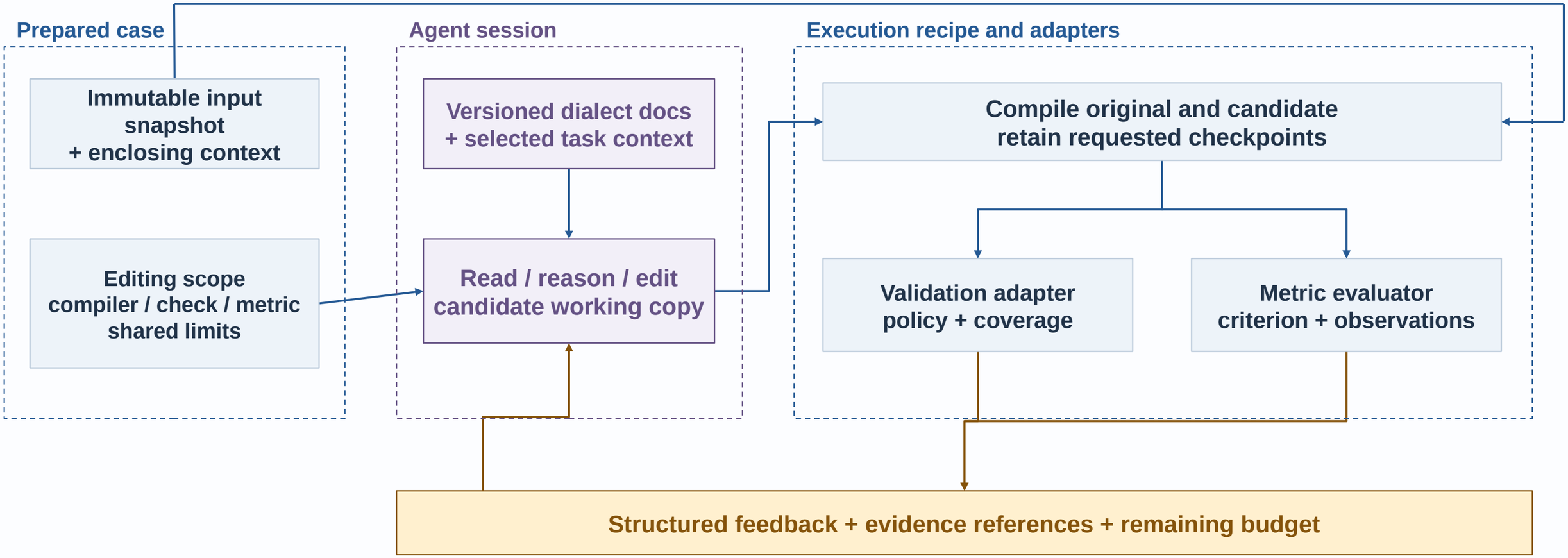
Screen uses instruction count or llvm-mca estimates; syntactic alternatives may also be retained.

PRAGMA: A Profiling-Reasoned Multi-Agent Framework for Automatic Kernel Optimization



Editable reconstructions of LPO Fig. 2 and PRAGMA Figs. 1–2. LLVM and CPU/GPU tool assumptions belong to the papers.

Proposed Ascend investigation architecture



Persist snapshots, candidates, operations, feedback delivery state and budget reservations across every stage.

Finding eligibility = required checks pass + selected criterion met + complete runnable bundle.

Candidate transformations and applicability conditions

Family	Candidate change	Conditions / checkpoint information
Arithmetic and clamps	Remove a clamp subsumed by later operations.	Integer width, signedness, overflow flags; exact sequence.
Predicates / selects	Remove a NaN guard before ordered equality.	FP predicate semantics; other uses of the sanitized value.
Memory sequences	Adjacent narrow loads → one wider load.	Bounds, alignment, endianness, memory space and effects.
Vectorization / lowering	Restructure conditions to expose target instructions.	Vector width, legal operations, contraction permissions.
Reduction organization	Change reduction structure or target mapping.	Shape, neutral element, NaN handling, numeric order.
Layout conversions	Eliminate an inverse pair or retain one layout.	Physical layout, aliasing and requirements of consumers.

LPO motivates the first three families. PRAGMA motivates vectorization/reduction. Layout conversion is our additional candidate.

MLIR case — redundant NaN handling

Shared operands %x : f32 %zero = 0.0 : f32 %one = 1.0 : f32

Original

```
%ordinary = arith.cmpf ord, %x, %x : f32
%safe = arith.select %ordinary, %x, %zero : f32
%equal = arith.cmpf oeq, %safe, %one : f32
```

Candidate

```
%equal = arith.cmpf oeq, %x, %one : f32
```

Input class	Original result	Candidate result
NaN	select zero; 0.0 == 1.0 → false	ordered equality → false
Ordinary value	x == 1.0	x == 1.0

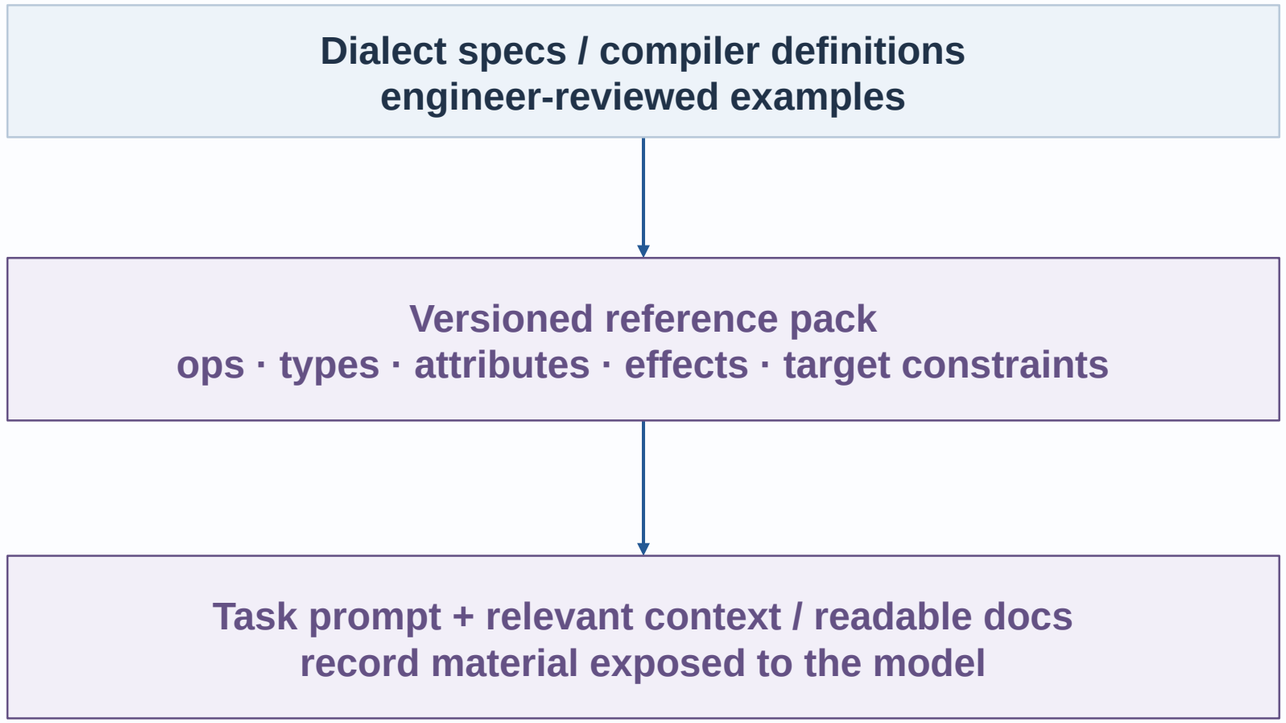
LPO Figure 4(c,f), adapted to MLIR. Inspect post-continuation output and the selected checks before retaining a finding.

Request resolution and dialect knowledge delivery

Illustrative request shape

```
input: module.mlir
mode: specified_range
region: operation_range
checkpoint: input_profile
pipeline: continuation_profile
checking: validation_profile
objective: evaluator_profile
limits: request_budget
```

Defaults → request → per-input overrides
Shared request limits stay upper bounds.



Prompt/context study: inspect public harnesses, then compare direct dialect context, retrieval and added analyses.

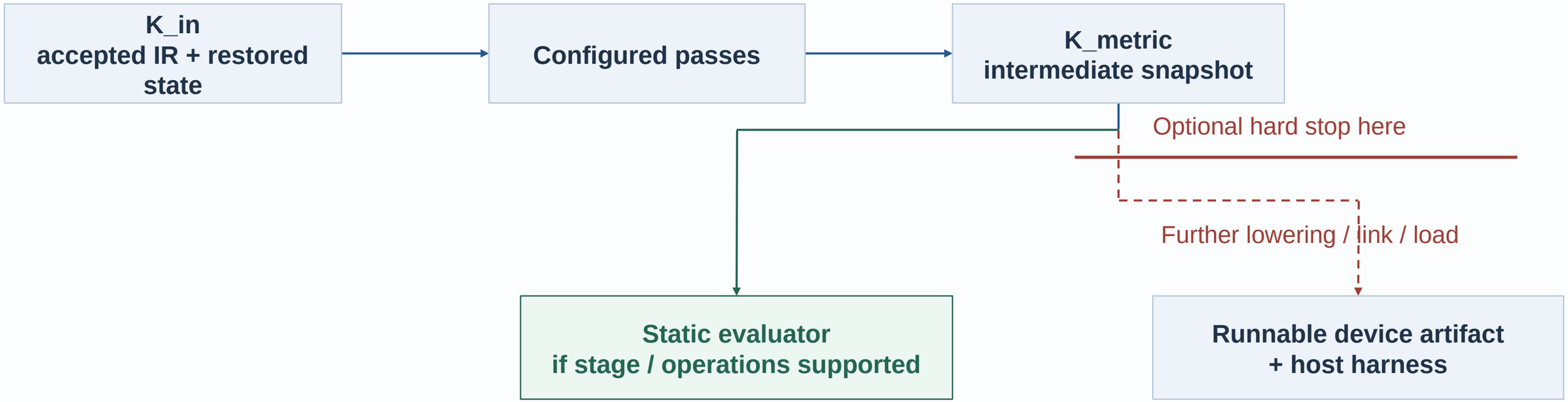
Agent context research: public harness starting points

Reference	Public mechanism to inspect	Possible Ascend experiment
Codex	Published starter prompt, repository instructions and tool use.	How to combine task instructions, dialect facts and compiler tools.
Claude Code	Custom subagent system prompts and scoped tool access.	Explicit IR-review role with a controlled tool set.
OpenCode	Agent definitions with custom prompts and tool permissions.	Separate reusable domain instructions from per-case context.
LangChain SQL agent	Schema retrieval and correction from database errors.	Retrieve operation semantics on demand and repair from diagnostics.
PaperQA	Evidence extraction and answer-synthesis prompt templates.	Summarize relevant dialect documentation while retaining sources.

Baseline: supplied IR + versioned dialect references. Compare direct context, retrieved documentation and targeted analyses under the same model and search budget.

Compilation checkpoints determine route compatibility

Illustrative continuation for both original and candidate



Runtime validation needs a runnable artifact. A hard stop before its required lowering blocks this policy. The metric checkpoint can remain earlier if continuation is allowed.

Route study output: checkpoint catalog, validation coverage and a demonstrated / incompatible / unknown configuration matrix.

Validation research: LLVM and MLIR starting points

Starting point	Input / available evidence	What the study must establish
MLIR verifier / project driver	Registered dialect IR and operation invariants.	Parsing / verification / compilation success is structural evidence.
Alive2 (LLVM)	LLVM IR original/candidate refinement check.	A usable LLVM route and supported semantics. Custom lowering needs separate justification.
MLIR-TV	SMT-based translation validation for supported MLIR.	Supported operations, shapes, memory and FP semantics for selected cases.
xdsl-smt / xdsl-tv	Refinement checks for subsets of arith, comb and func.	Representation mapping and the cost of missing operation semantics.
Execution comparison	Project harness, recorded inputs and observable outputs.	Runnable route, NPU requirements, tolerances and test coverage.

Retain a finding when the selected checks pass, the metric criterion is met and the replay bundle is complete. Record failed, unsupported and inconclusive checks separately.

Metric research: concrete indicators and tools

Candidate metric	Starting point / resources	Interpretation and open checks
Operation counts	MLIR print-op-stats at a fixed checkpoint.	Cheap structural screen. Later lowering may erase or reverse the reduction.
Logical memory work	Shape/layout-based access and byte estimates.	Proposed indicator. Reuse, transfers and physical layout affect real traffic.
Target cost estimates	LLVM TTI for supported IR targets. llvm-mca for supported CPU assembly.	Study target / operation coverage. An equivalent Ascend model is not established.
Kernel time / counters	Ascend msProf / msprof op with a runnable operator.	Recorded durations / counter data. Check local tooling, warm-up, repeats and collection cost.



Metric improvement and preserved behavior require separate evidence. Static indicators alone do not establish runtime speedup.

Feedback, stopping rules and crash recovery

Observed outcome	Feedback / next action
Compilation rejection or behavior mismatch	Send diagnostics or failing input with original/candidate identities. Agent may repair.
No qualifying improvement	Send metric values, stage, units, criterion and evidence references.
Unsupported check / unavailable hardware	Expose coverage or infrastructure cause; retain an unresolved operation.

Durable operation protocol

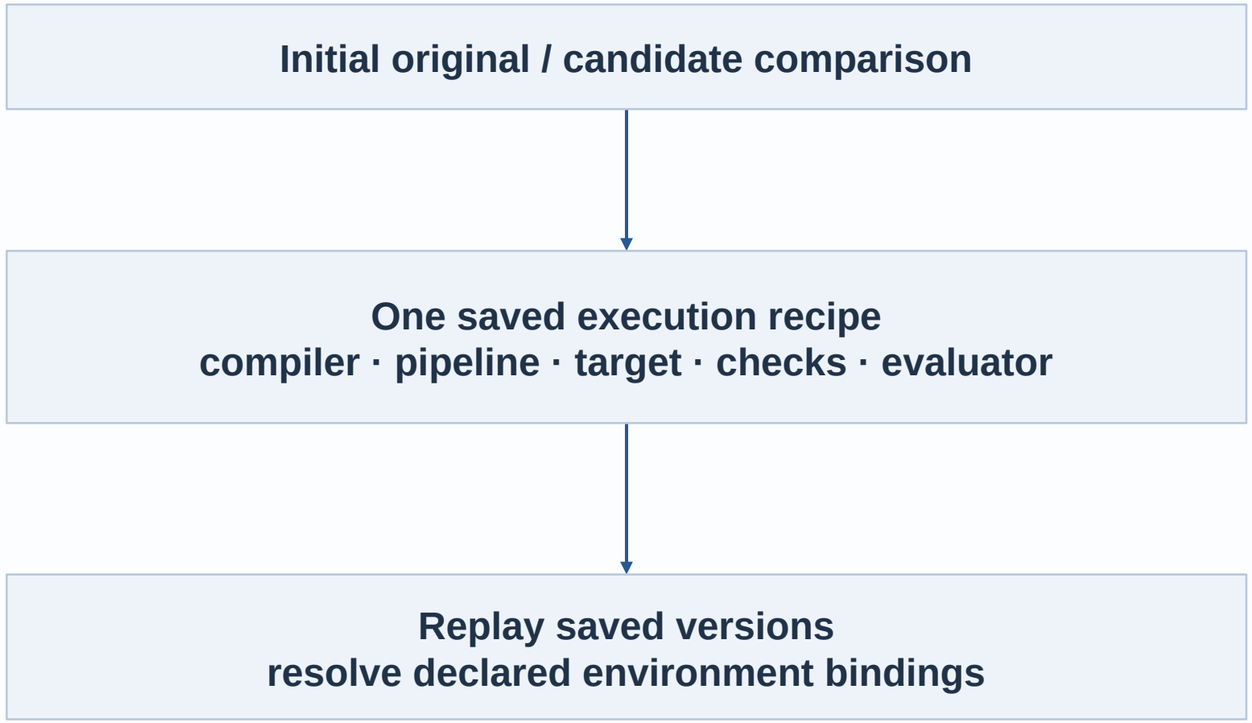


Restart → reconcile in-flight work → reuse completed stages → restore or reconstruct agent context → continue with recorded budget.

Preserve each submitted candidate and the best eligible finding independently of the current working file.

The finding is a replayable experiment

```
finding-ID/
  report.md
  manifest.json
  original/
  candidate/
  changes.diff
  evidence/
  reproduce/
  replays/
```



Proposed interface

```
framework replay ./finding-ID
```

Each replay stores fresh check results, metric samples and resource usage linked to the original finding.

Constraints resolve into an explicit experiment profile

Constraint	Resolved setting	Failure or boundary to expose
Editing scope	Whole artifact or complete operation range; context outside scope read-only.	Out-of-scope edits reject the candidate.
Semantics	Types, shapes, layouts, effects, aliasing, FP behavior and tolerances.	Additional agent assumptions need evidence.
Compilation	Start checkpoint, compiler build, pass configuration, outputs and hard stop.	Unsupported entry or incompatible continuation.
Evidence	Required checks; metric stage, units, target and improvement threshold.	Unsupported coverage and inconclusive outcomes.
Resources	Candidates, shared request limits, model calls/tokens and host/NPU jobs.	Stop scheduling beyond the remaining budget.

MVP scope and implementation stages

One engineer-supplied input mode, one demonstrated compilation route, one checking policy and one evaluator.

Stage	Deliverable
Month 1	Establish compiler entry / continuation; direct agent experiments on real cases.
Months 2–3	Implement profiles, snapshots, dialect pack, agent edits, runners, feedback and durable state.
Month 4 · MVP	Complete budget enforcement, interrupted-work resume, finding export and relocated replay.
Month 5	Evaluate held-out cases with agreed usefulness criteria, repeats and resource budgets.
Month 6	Implement and compare one context, tool, discovery or feedback refinement.

Resources — one developer, part-time compiler expertise, model/host capacity and NPU access when the selected route requires it.

Primary risks concern finding yield and integration effort

Too few useful findings

Rewrites may be invalid, trivial, narrowly applicable or already handled by a later compiler stage.

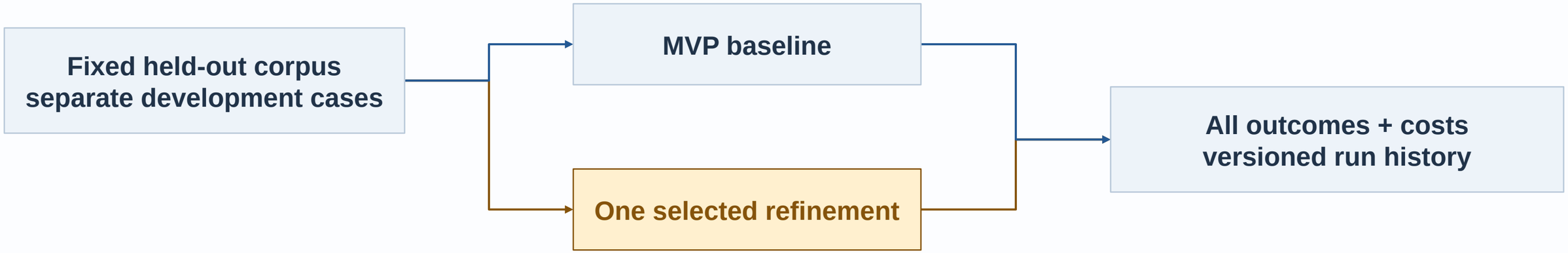
Integration costs dominate

Saved IR may require state restoration, custom harnesses, checking coverage or scarce device access.

Additional risk	Failure to watch for
Expert effort	Preparation, reviewing assumptions and maintaining the harness consume the time saved.
Metric relevance	A lower score fails to improve relevant downstream code or workloads.
Correctness coverage	Selected checks miss behavior changed by the rewrite.

Useful findings are the primary outcome. Effort saving and evidence limits are measured alongside finding quality.

Evaluation protocol and controlled refinement studies



Primary evidence	Additional outcomes
Engineer-reviewed useful findings	Rejected, duplicate, unsupported and inconclusive cases.
Replays of retained findings	Coverage, check outcomes and metric changes per replay.
At least one real case for MVP	Preparation/review effort, model use and host/NPU usage.

Before the study, agree case selection, usefulness criteria, search budgets and repeats. Any effort-saving claim identifies its reference workflow.